

**UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
FACULTAD DE INGENIERÍA EN CIENCIAS DE LA
COMPUTACIÓN Y TELECOMUNICACIONES**



**GESTIÓN DIGITAL DE ALQUILER DE VEHÍCULOS MEDIANTE UNA BASE DE
DATOS CENTRALIZADA.**

"MOVISHARE"

PROYECTO# 133

CARRERA: ING. INFORMATICA

MATERIA:

INF112- BASE DATOS I

DOCENTE:

ING. JUAN CARLOS PEINADO PEREIRA

AREA-CATEGORIA:

DESARROLLO DE SOFTWARE – INTERMEDIO

AUTORES:

ORCID:

SAHONERO SALAS SARA RAQUEL

0009-0003-1367-1417

VINCENTY MAMANI DAVID LEONEL

0009-0002-5967-6024

PADILLA FLORES JARITH

0009-0003-89371851

ORELLANA GUTIERREZ DIEGO

0009-0009-6132-562X

NEIL SERRANO VARGAS

0009-0006-5661-0192

INDICE

1	RESUMEN	1
2	INTRODUCCIÓN	2
2.1	Contexto General	2
2.2	Problemática Identificada.....	2
2.3	Qué es MOVISHARE	3
2.4	Delimitación del Proyecto	4
3	MARCO TEÓRICO	4
3.1	Sistemas de Gestión Empresarial	4
3.2	Gestión de Flotas Vehiculares	5
3.3	Bases de Datos Relacionales	5
3.4	Principios ACID.....	5
3.5	Normalización hasta 3NF	6
3.6	Seguridad de la Información	6
3.7	Metodología Scrum	7
3.8	Arquitectura de Tres Capas	8
3.9	Sistemas de Información Empresarial	8
3.10	DISEÑO DE BASE DE DATOS	10
3.11	Entidades del Sistema	11
3.12	Diagrama Entidad-Relación (DER).....	11
3.13	Relaciones entre Entidades.....	14
3.14	Modelo Relacional — SQL Completo	14
3.15	Normalización 3NF	20
4	HIPÓTESIS Y SUPUESTOS	21
4.1	Hipótesis Principal	21
4.2	Sub-Hipótesis Específicas	21
4.3	Supuestos de Diseño	22
5	OBJETIVOS.....	22
5.1	Objetivo General	22
5.2	Objetivos Específicos	23
6	METODOLOGÍA	23
6.1	Tipo y Enfoque de Investigación.....	23
6.2	Instrumento de Recolección	24
6.3	Metodología de Desarrollo — Scrum.....	24
6.4	Roles en el Proyecto.....	25

6.5	CONTEXTO LEGAL Y ECONÓMICO	25
6.5.1	Marco Legal del Alquiler Vehicular en Bolivia.....	25
□	Código Civil Boliviano — Contrato de Arrendamiento (Art. 685 y 1297):.....	25
6.6	Mercado de Alquiler Vehicular en Santa Cruz de la Sierra	26
6.7	Sprints	28
6.8	ARQUITECTURA TÉCNICA.....	28
6.8.1	Modelo de Arquitectura.....	28
6.8.2	¿Por qué arquitectura de tres capas?	29
6.9	Stack Tecnológico.....	29
6.10	Patrón de Diseño — MVC	30
6.11	Roles del Sistema y Control de Acceso	31
6.12	ANÁLISIS DE RIESGOS Y SEGURIDAD	32
6.12.2	Riesgos Técnicos.....	33
6.12.3	Riesgos Operativos.....	35
6.13	IMPACTO ESPERADO	35
6.13.1	Impacto Operativo.....	35
6.13.2	Impacto Económico.....	37
6.13.3	Impacto en la Seguridad de la Información	37
6.13.4	Impacto Académico.....	38
7	CONCLUSIONES.....	38
7.1	Conclusiones Generales	38
7.2	Limitaciones del Proyecto	39
7.3	Trabajo Futuro	39
8	RECOMENDACIONES.....	40
8.1	Para el Desarrollo del Sistema.....	40
8.2	Para la Empresa que Adopte el Sistema	40
8.3	Para Futuras Versiones del Sistema	41
9	BIBLIOGRAFÍA.....	43
9.1	Libros y Textos Académicos.....	43
9.2	Documentación Técnica Oficial	44
9.3	Marco Legal	44

1 RESUMEN

MOVISHARE es un sistema de escritorio diseñado para facilitar la gestión de alquiler de vehículos en empresas del rubro ubicadas en el área urbana de Santa Cruz de la Sierra, Bolivia. Su propósito es reemplazar los procesos manuales registros en papel, hojas de cálculo improvisadas, control verbal de disponibilidad por una herramienta digital centralizada, confiable y fácil de usar.

El sistema permite registrar vehículos, gestionar clientes, crear y controlar reservas, registrar pagos y generar reportes, todo desde una interfaz de escritorio con acceso restringido según el rol del usuario. Cada operación queda registrada automáticamente, eliminando errores humanos y garantizando que la información esté siempre disponible y ordenada.

El desarrollo sigue la metodología Scrum, lo que permite entregar avances verificables por etapas y adaptarse a observaciones durante el proceso. La viabilidad del sistema se evalúa mediante pruebas con usuarios reales, con criterios de usabilidad y rendimiento claramente definidos.

2 INTRODUCCIÓN

2.1 Contexto General

Santa Cruz de la Sierra es la ciudad más poblada de Bolivia y su principal centro económico. Su crecimiento sostenido ha impulsado el desarrollo de diversos sectores comerciales, entre ellos el alquiler de vehículos, una actividad que responde a la demanda de movilidad flexible por parte de empresas, turistas, estudiantes y residentes que por distintas razones no disponen de vehículo propio o necesitan uno adicional de forma temporal.

Sin embargo, muchas empresas dedicadas a este rubro en la ciudad operan aún de forma manual o con herramientas genéricas como hojas de cálculo, lo que genera problemas concretos en el día a día: reservas mal registradas, vehículos alquilados sin control de disponibilidad, cobros incorrectos, pérdida de historial de clientes y ausencia total de trazabilidad sobre quién hizo qué dentro de la empresa.

2.2 Problemática Identificada

El problema que motiva el desarrollo de MOVISHARE tiene dos dimensiones:

Desde la operación interna de la empresa:

Una empresa de alquiler vehicular que no cuenta con un sistema especializado enfrenta diariamente situaciones como asignar un vehículo que ya fue reservado, calcular manualmente el precio de cada alquiler con riesgo de error, no saber en tiempo real qué vehículos están disponibles, en mantenimiento o dañados, y no tener forma de saber qué acciones realizó cada empleado durante el día.

Desde la atención al cliente:

Un cliente que llega a la empresa espera ser atendido con rapidez y precisión. Sin un sistema,

el empleado debe buscar en cuadernos o archivos de Excel, calcular fechas y precios manualmente y confirmar disponibilidad preguntando a sus compañeros. Ese proceso es lento, propenso a errores y genera una experiencia negativa para el cliente.

2.3 Qué es MOVISHARE

MOVISHARE es un sistema de escritorio que centraliza toda la operación de una empresa de alquiler de vehículos en una sola herramienta. Desde el registro de cada vehículo de la flota hasta la generación de reportes mensuales, pasando por la gestión de clientes, reservas, pagos y mantenimientos.

El sistema está diseñado para dos tipos de usuarios dentro de la empresa: el administrador, que tiene acceso total al sistema incluyendo configuración, reportes y auditoría, y el empleado, que gestiona la operación diaria atención al cliente, registro de reservas, cobro de pagos y control de devoluciones.

MOVISHARE no es una aplicación web ni una app móvil. Es un sistema instalado directamente en los equipos de la empresa, lo que lo hace independiente de la conexión a internet y adecuado para el contexto operativo de pequeñas y medianas empresas de alquiler en Santa Cruz de la Sierra.

2.4 Delimitación del Proyecto

Dimensión	Alcance
Espacial	Área urbana de Santa Cruz de la Sierra, Bolivia
Temática	Gestión de flota, clientes, reservas, pagos y mantenimiento
Usuarios	Administrador y empleado de la empresa
Plataforma	Sistema de escritorio (no web, no móvil)
Tipo de empresa	Empresa con flota propia que alquila a clientes externos

3 MARCO TEÓRICO

3.1 Sistemas de Gestión Empresarial

Un sistema de gestión empresarial es una herramienta de software diseñada para centralizar, organizar y automatizar los procesos operativos de una organización. Su propósito principal es reemplazar tareas manuales repetitivas por procesos digitales controlados, reduciendo errores humanos y mejorando la eficiencia del trabajo diario.

En el contexto de pequeñas y medianas empresas, estos sistemas no necesitan ser complejos ni costosos. Lo que importa es que respondan directamente a los procesos reales del negocio en el caso de MOVISHARE, los procesos de una empresa de alquiler vehicular: registrar vehículos, atender clientes, gestionar reservas y controlar pagos.

3.2 Gestión de Flotas Vehiculares

La gestión de flotas se refiere al conjunto de procesos mediante los cuales una empresa controla el ciclo de vida de sus vehículos: disponibilidad, asignación, mantenimiento y estado general. Una flota bien gestionada reduce los tiempos muertos, previene conflictos de asignación y permite tomar decisiones basadas en información real.

Sin un sistema de gestión, el control de flota depende de la memoria y los apuntes de los empleados, lo que inevitablemente genera errores. Con MOVISHARE, el estado de cada vehículo disponible, alquilado, en mantenimiento o dañado se actualiza automáticamente con cada operación registrada en el sistema.

3.3 Bases de Datos Relacionales

Una base de datos relacional organiza la información en tablas estructuradas que se relacionan entre sí mediante claves. Este modelo, propuesto por Edgar F. Codd en 1970, es el más utilizado en sistemas de gestión empresarial por su capacidad para mantener la integridad de los datos y responder consultas complejas de forma eficiente.

En MOVISHARE, la base de datos relacional permite, por ejemplo, consultar todas las reservas activas de un cliente específico, o saber cuántos días lleva en mantenimiento un vehículo determinado, cruzando información de múltiples tablas en una sola consulta.

3.4 Principios ACID

Todo sistema de base de datos relacional confiable debe garantizar las propiedades ACID en cada transacción:

Propiedad	Significado	Aplicación en MOVISHARE
Atomicidad	Una operación se completa entera o no se completa	Si falla el registro de una reserva, ningún dato queda guardado a medias
Consistencia	Los datos siempre pasan de un estado válido a otro válido	No puede existir una reserva sin cliente o sin vehículo asociado
Aislamiento	Dos operaciones simultáneas no se interfieren	Dos empleados no pueden reservar el mismo vehículo al mismo tiempo
Durabilidad	Una vez confirmada, la operación persiste aunque falle el sistema	Un pago registrado no desaparece si se corta la luz

3.5 Normalización hasta 3NF

La normalización es el proceso de organizar las tablas de una base de datos para eliminar redundancias y evitar errores al insertar, actualizar o eliminar datos. MOVISHARE aplica normalización hasta la Tercera Forma Normal (3NF), que garantiza:

- 1NF: cada celda contiene un único valor, sin listas ni grupos repetitivos.
- 2NF: todos los atributos dependen completamente de la clave primaria.
- 3NF: ningún atributo depende de otro atributo que no sea la clave primaria.

3.6 Seguridad de la Información

La seguridad de la información en un sistema de gestión empresarial abarca tres dimensiones fundamentales:

- Confidencialidad: solo las personas autorizadas acceden a la información. En MOVISHARE esto se garantiza mediante autenticación con usuario y contraseña encriptada con bcrypt, y control de acceso por roles.

- **Integridad:** los datos no pueden ser modificados o eliminados sin autorización. En MOVISHARE esto se garantiza mediante restricciones en la base de datos (FK, CHECK, UNIQUE) y un módulo de auditoría que registra cada acción del sistema.
- **Disponibilidad:** el sistema debe estar accesible cuando se necesita. En MOVISHARE esto se garantiza mediante backups automáticos diarios y un objetivo de uptime igual o superior al 95%.

El marco de referencia utilizado es el NIST Cybersecurity Framework, que organiza la seguridad en cinco funciones: Identificar, Proteger, Detectar, Responder y Recuperar, aplicadas en cada módulo del sistema según el nivel de riesgo correspondiente.

3.7 Metodología Scrum

Scrum es una metodología ágil de desarrollo de software que organiza el trabajo en ciclos cortos llamados sprints, cada uno con un objetivo claro y un entregable funcional al final. Fue elegida para el desarrollo de MOVISHARE por tres razones principales:

- **Adaptabilidad:** permite incorporar observaciones del tutor o cambios de requisitos entre sprints sin afectar el trabajo ya entregado.
- **Verificabilidad:** cada sprint produce un módulo funcional que puede ser revisado y aprobado antes de continuar con el siguiente.
- **Trazabilidad:** el Product Backlog con sus historias de usuario documenta qué se planificó, qué se construyó y qué quedó pendiente, generando evidencia de proceso para el tribunal académico.

Los tres artefactos principales de Scrum utilizados en MOVISHARE son el Product Backlog (lista priorizada de 28 historias de usuario en 6 épicas), el Sprint Backlog (historias

seleccionadas para cada sprint) y el Incremento (módulo funcional entregado al final de cada sprint).

3.8 Arquitectura de Tres Capas

La arquitectura de tres capas es un patrón de diseño de software que separa el sistema en tres niveles independientes:

- Capa de presentación: lo que el usuario ve e interactúa — formularios, botones, tablas, mensajes.
- Capa de lógica de negocio: las reglas que gobiernan el sistema — validaciones, cálculos, control de acceso.
- Capa de datos: el almacenamiento y recuperación de información — MySQL, consultas SQL, transacciones.

Esta separación hace que el sistema sea más fácil de mantener, probar y modificar, ya que un cambio en la interfaz no afecta la lógica ni la base de datos, y viceversa.

3.9 Sistemas de Información Empresarial

Un sistema de información es un conjunto organizado de personas, procesos, datos y tecnología que trabajan juntos para recolectar, procesar, almacenar y distribuir información útil para la toma de decisiones dentro de una organización.

Los sistemas de información empresarial se clasifican según su función:

Tipo	Función	Ejemplo
TPS (Transaction Processing System)	Registra operaciones del día a día	Registrar una reserva, un pago
MIS (Management Information System)	Genera reportes para decisiones gerenciales	Reporte de ingresos mensuales
DSS (Decision Support System)	Apoya decisiones complejas con análisis	Análisis de vehículos más rentables

MOVISHARE opera principalmente como un **TPS** su función central es registrar y gestionar las transacciones diarias de la empresa con elementos de **MIS** en su módulo de reportes, donde el administrador puede consultar información consolidada para tomar decisiones sobre la flota y la operación.

Esta clasificación ubica a MOVISHARE dentro de la teoría formal de sistemas de información, dándole respaldo académico más allá de ser simplemente "una aplicación de escritorio".

3.10 DISEÑO DE BASE DE DATOS

La base de datos es el núcleo del sistema MOVISHARE. Toda la información que el sistema genera, consulta y administra — vehículos, clientes, reservas, pagos, mantenimientos y acciones de usuarios — reside en ella. Un diseño correcto garantiza que esa información sea confiable, consistente y recuperable en cualquier momento.

El modelo de datos de MOVISHARE está compuesto por 12 tablas relacionadas entre sí, diseñadas siguiendo el modelo relacional de Codd y normalizadas hasta la Tercera Forma Normal (3NF). El sistema gestor de base de datos utilizado es MySQL 8.0+, administrado visualmente mediante MySQL Workbench. La base de datos se denomina movishare_db y utiliza el charset utf8mb4 con collation utf8mb4_general_ci para soporte completo de caracteres especiales.

3.11 Entidades del Sistema

Antes de construir el diagrama, se identificaron las entidades que representan los objetos principales del negocio. MOVISHARE utiliza un patrón de herencia mediante la entidad PERSONA como tabla base para USUARIO y CLIENTE, evitando duplicación de datos comunes (CI, nombre, teléfono, dirección):

Entidad	Representa	Relación principal
PERSONA	Datos base compartidos entre usuarios y clientes	Tabla padre de USUARIO y CLIENTE
ROL	Catálogo de roles del sistema	Clasifica usuarios (ADMIN, OPERADOR, TÉCNICO)
USUARIO	Empleados y administradores del sistema	Hereda de PERSONA, registra reservas y pagos
CLIENTE	Personas que alquilan vehículos	Hereda de PERSONA, realiza reservas
LICENCIA	Licencia de conducir del cliente	Asociada a un cliente
CATEGORIA	Categoría del vehículo con tarifa y seguro	Clasifica automóviles y define precios
AMBITO_USO	Ámbito geográfico del alquiler con costo adicional	Asociado a cada reserva
AUTOMOVIL	Vehículos de la flota de la empresa	Es asignado en reservas, tiene mantenimientos
RESERVA	Contrato de alquiler entre cliente y empresa	Conecta cliente, vehículo, usuario y ámbito
RESERVA_AUTOMOVIL	Detalle operativo de la reserva (gasolina, km)	Asigna un vehículo a una reserva
PAGO	Registro del cobro de una reserva	Asociado a una reserva
MANTENIMIENTO	Registro de mantenimientos de cada vehículo	Asociado a un vehículo
AUDITORIA	Registro de todas las acciones del sistema	Asociada a un usuario

3.12 Diagrama Entidad-Relación (DER)

El DER representa visualmente las entidades, sus atributos y las relaciones entre ellas.

Se utilizan dos representaciones complementarias:

muestra las entidades como rectángulos, los atributos como óvalos, las relaciones como rombos y las cardinalidades en las conexiones. Incluye las 12 entidades del sistema con sus atributos, claves primarias subrayadas y relaciones identificadas.

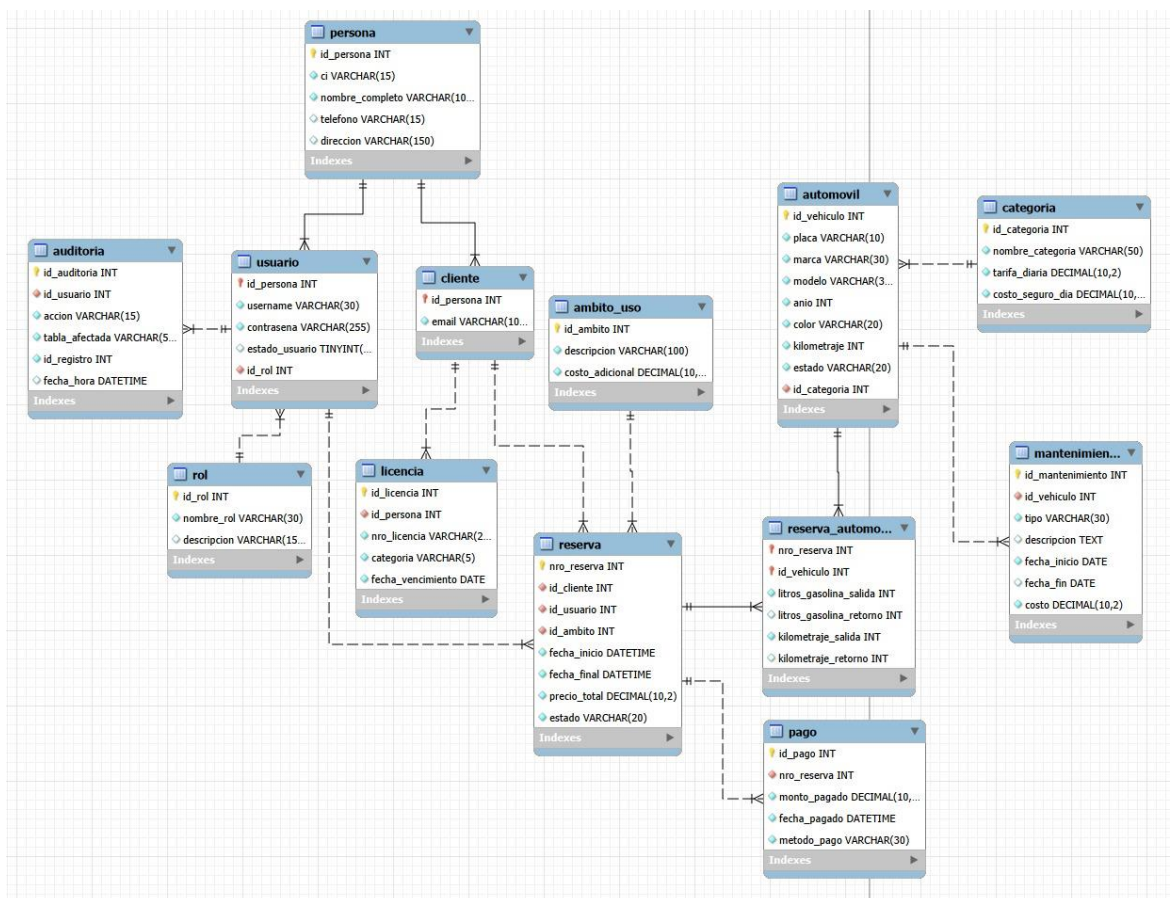


Figura 2 — Modelo Relacional (notación UML/MySQL Workbench):

muestra las tablas con sus campos, tipos de datos, claves primarias (PK), claves foráneas (FK) e índices. Generado directamente desde la base de datos implementada en MySQL 8.0.

Nota: Ambos diagramas están incluidos como figuras separadas en el documento principal. El diagrama UML fue exportado desde MySQL Workbench y refleja exactamente la estructura implementada en el script `schema.sql`.

3.13 Relaciones entre Entidades

Entidad A	Relación	Entidad B	Cardinalidad
PERSONA	Hereda en	USUARIO	1:1 — una persona puede ser un usuario
PERSONA	Hereda en	CLIENTE	1:1 — una persona puede ser un cliente
ROL	Clasifica	USUARIO	1:N — un rol clasifica muchos usuarios
CLIENTE	Tiene	LICENCIA	1:N — un cliente tiene una o más licencias
CLIENTE	Genera	RESERVA	1:N — un cliente genera muchas reservas
USUARIO	Registra	RESERVA	1:N — un usuario registra muchas reservas
AMBITO_USO	Aplica en	RESERVA	1:N — un ámbito se aplica a muchas reservas
RESERVA	Detalla en	RESERVA_AUTOMOVIL	1:1 — cada reserva tiene un detalle operativo
AUTOMOVIL	Asignado en	RESERVA_AUTOMOVIL	1:N — un vehículo aparece en muchas reservas
CATEGORIA	Clasifica	AUTOMOVIL	1:N — una categoría clasifica muchos vehículos
RESERVA	Genera	PAGO	1:N — una reserva puede tener varios pagos
AUTOMOVIL	Tiene	MANTENIMIENTO	1:N — un vehículo tiene muchos mantenimientos
USUARIO	Genera	AUDITORIA	1:N — cada acción del usuario genera un log

3.14 Modelo Relacional — SQL Completo

A continuación se presenta el script DDL completo que crea la base de datos, las 12 tablas con sus restricciones, y los índices estratégicos para rendimiento. Este script corresponde exactamente al archivo db/schema.sql del repositorio.

Creación de la Base de Datos

```
CREATE DATABASE movishare_db  
  
CHARACTER SET utf8mb4  
  
COLLATE utf8mb4_general_ci;  
  
USE movishare_db;
```

TABLA: ROL

```
CREATE TABLE rol (  
    id_rol      INT AUTO_INCREMENT PRIMARY KEY,  
    nombre_rol  VARCHAR(50) NOT NULL UNIQUE,  
    descripcion VARCHAR(255)  
) ENGINE=InnoDB;
```

TABLA: CATEGORIA

```
CREATE TABLE categoria (  
    id_categoria  INT AUTO_INCREMENT PRIMARY KEY,  
    nombre_categoria VARCHAR(50) NOT NULL UNIQUE,  
    tarifa_diaria  DECIMAL(12,2) NOT NULL DEFAULT 0.00,  
    costo_seguro_dia DECIMAL(12,2) NOT NULL DEFAULT 0.00  
) ENGINE=InnoDB;
```

TABLA: AMBITO_USO

```
CREATE TABLE ambito_uso (  
    id_ambito      INT AUTO_INCREMENT PRIMARY KEY,  
    descripcion     VARCHAR(100) NOT NULL,  
    costo_adicional DECIMAL(12,2) NOT NULL DEFAULT 0.00  
) ENGINE=InnoDB;
```

TABLA: PERSONA

```
CREATE TABLE persona (  
    id_persona  INT AUTO_INCREMENT PRIMARY KEY,  
    ci          VARCHAR(20) NOT NULL,  
    nombre_completo VARCHAR(100) NOT NULL,  
    telefono    VARCHAR(20),  
    direccion   VARCHAR(150)
```

```
) ENGINE=InnoDB;
```

TABLA: USUARIO

```
CREATE TABLE usuario (  
  id_persona    INT PRIMARY KEY,  
  username      VARCHAR(50) NOT NULL UNIQUE,  
  contrasena    VARCHAR(255) NOT NULL,  
  estado_usuario ENUM('ACTIVO','INACTIVO','BLOQUEADO')  
                NOT NULL DEFAULT 'ACTIVO',  
  id_rol        INT NOT NULL,  
  FOREIGN KEY (id_persona)  
    REFERENCES persona(id_persona) ON DELETE CASCADE,  
  FOREIGN KEY (id_rol)  
    REFERENCES rol(id_rol)  
) ENGINE=InnoDB;
```

TABLA: CLIENTE

```
CREATE TABLE cliente (  
  id_persona    INT PRIMARY KEY,  
  email         VARCHAR(100) NOT NULL,  
  FOREIGN KEY (id_persona)  
    REFERENCES persona(id_persona) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

TABLA: LICENCIA

```
CREATE TABLE licencia (  
  id_licencia    INT AUTO_INCREMENT PRIMARY KEY,  
  id_persona     INT NOT NULL,  
  nro_licencia   VARCHAR(50) NOT NULL,  
  categoria      VARCHAR(10) NOT NULL,  
  fecha_vencimiento DATE NOT NULL,  
  FOREIGN KEY (id_persona)  
    REFERENCES cliente(id_persona) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

TABLA: AUTOMOVIL

```
CREATE TABLE automovil (  
    id_vehiculo INT AUTO_INCREMENT PRIMARY KEY,  
    placa VARCHAR(15) NOT NULL UNIQUE,  
    marca VARCHAR(50) NOT NULL,  
    modelo VARCHAR(50) NOT NULL,  
    anio INT NOT NULL,  
    color VARCHAR(30),  
    kilometraje DECIMAL(12,2) NOT NULL DEFAULT 0.00,  
    estado ENUM('DISPONIBLE','ALQUILADO',  
        'MANTENIMIENTO','INACTIVO')  
        NOT NULL DEFAULT 'DISPONIBLE',  
    id_categoria INT NOT NULL,  
    FOREIGN KEY (id_categoria)  
        REFERENCES categoria(id_categoria)  
) ENGINE=InnoDB;
```

TABLA: MANTENIMIENTO

```
CREATE TABLE mantenimiento (  
    id_mantenimiento INT AUTO_INCREMENT PRIMARY KEY,  
    id_vehiculo INT NOT NULL,  
    tipo VARCHAR(50) NOT NULL,  
    descripcion VARCHAR(255),  
    fecha_inicio DATE NOT NULL,  
    fecha_fin DATE,  
    costo DECIMAL(12,2) NOT NULL DEFAULT 0.00,  
    FOREIGN KEY (id_vehiculo)  
        REFERENCES automovil(id_vehiculo) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

TABLA: RESERVA

```
CREATE TABLE reserva (  
    nro_reserva INT AUTO_INCREMENT PRIMARY KEY,  
    id_cliente INT NOT NULL,  
    id_usuario INT NOT NULL,
```

```

id_ambito    INT NOT NULL,
fecha_inicio DATE NOT NULL,
fecha_final  DATE NOT NULL,
precio_total DECIMAL(12,2) NOT NULL DEFAULT 0.00,
estado       ENUM('PENDIENTE','ACTIVA',
                  'COMPLETADA','CANCELADA')
              NOT NULL DEFAULT 'PENDIENTE',
FOREIGN KEY (id_cliente)
              REFERENCES cliente(id_persona),
FOREIGN KEY (id_usuario)
              REFERENCES usuario(id_persona),
FOREIGN KEY (id_ambito)
              REFERENCES ambito_uso(id_ambito)
) ENGINE=InnoDB;

```

TABLA: RESERVA_AUTOMOVIL

```

CREATE TABLE reserva_automovil (
nro_reserva    INT PRIMARY KEY,
id_vehiculo    INT NOT NULL,
litros_gasolina_salida  DECIMAL(8,2) DEFAULT 0.00,
litros_gasolina_retorno DECIMAL(8,2) DEFAULT 0.00,
kilometraje_salida    DECIMAL(12,2) DEFAULT 0.00,
kilometraje_retorno   DECIMAL(12,2) DEFAULT 0.00,
FOREIGN KEY (nro_reserva)
              REFERENCES reserva(nro_reserva) ON DELETE CASCADE,
FOREIGN KEY (id_vehiculo)
              REFERENCES automovil(id_vehiculo)
) ENGINE=InnoDB;

```

TABLA: PAGO

```
CREATE TABLE pago (  
    id_pago      INT AUTO_INCREMENT PRIMARY KEY,  
    nro_reserva  INT NOT NULL,  
    monto_pagado DECIMAL(12,2) NOT NULL DEFAULT 0.00,  
    fecha_pagado DATE NOT NULL,  
    metodo_pago  VARCHAR(30) NOT NULL,  
    FOREIGN KEY (nro_reserva)  
        REFERENCES reserva(nro_reserva) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

TABLA: AUDITORIA

```
CREATE TABLE auditoria (  
    id_auditoria INT AUTO_INCREMENT PRIMARY KEY,  
    id_usuario   INT,  
    accion       VARCHAR(50) NOT NULL,  
    tabla_afectada VARCHAR(50),  
    id_registro  INT,  
    fecha_hora   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

ÍNDICES ESTRATÉGICOS PARA RENDIMIENTO

```
CREATE INDEX idx_usuario_login  
    ON usuario(username, estado_usuario);  
CREATE INDEX idx_persona_nombre  
    ON persona(nombre_completo);  
CREATE INDEX idx_automovil_estado  
    ON automovil(estado);  
CREATE INDEX idx_mantenimiento_fechas  
    ON mantenimiento(fecha_inicio, fecha_fin);  
CREATE INDEX idx_reserva_fechas  
    ON reserva(fecha_inicio, fecha_final);  
CREATE INDEX idx_reserva_estado  
    ON reserva(estado);  
CREATE INDEX idx_pago_fecha
```

```

ON pago(fecha_pagado);

CREATE INDEX idx_auditoria_registro

ON auditoria(id_usuario, fecha_hora);

```

3.15 Normalización 3NF

Todas las tablas de MOVISHARE fueron verificadas y cumplen con las tres formas normales. La separación de PERSONA como tabla base, ROL y CATEGORIA como tablas catálogo, y LICENCIA como entidad independiente son decisiones de normalización que eliminan redundancias y dependencias transitivas:

Tabla	1NF	2NF	3NF	Observación
ROL	✓	✓	✓	Tabla catálogo simple, normalizada desde ENUM
CATEGORIA	✓	✓	✓	Tarifa y seguro dependen de la categoría, no del vehículo
AMBITO_USO	✓	✓	✓	Costo adicional depende del ámbito, no de la reserva
PERSONA	✓	✓	✓	Tabla base para herencia, sin dependencias transitivas
USUARIO	✓	✓	✓	Hereda de persona, rol como FK a tabla catálogo
CLIENTE	✓	✓	✓	Hereda de persona, solo atributos propios (email)
LICENCIA	✓	✓	✓	Separada de cliente para permitir múltiples licencias
AUTOMOVIL	✓	✓	✓	Categoría como FK elimina duplicación de tarifas
RESERVA	✓	✓	✓	Precio calculado al momento de inserción
RESERVA_AUTOMOVIL	✓	✓	✓	Detalle operativo con datos propios (gasolina, km)
PAGO	✓	✓	✓	Permite pagos parciales por reserva
MANTENIMIENTO	✓	✓	✓	Costo específico de cada mantenimiento
AUDITORIA	✓	✓	✓	Tabla de solo inserción, sin updates

4 HIPÓTESIS Y SUPUESTOS

4.1 Hipótesis Principal

"La implementación de un sistema de escritorio especializado para la gestión de alquiler de vehículos mejora significativamente la eficiencia operativa de la empresa, reduciendo errores en el registro de reservas, optimizando el control de disponibilidad de la flota y agilizando la atención al cliente, en comparación con los métodos manuales actualmente utilizados en empresas del rubro en el área urbana de Santa Cruz de la Sierra."

Esta hipótesis plantea que el problema no es la falta de capacidad de las personas sino la ausencia de una herramienta adecuada. MOVISHARE es esa herramienta.

4.2 Sub-Hipótesis Específicas

ID	Hipótesis	Variable central	Método de validación
H1	El sistema reduce el tiempo de registro de una reserva en al menos un 60% respecto al proceso manual	Tiempo de operación	Medición cronometrada pre/post sistema
H2	El sistema elimina los errores de doble reserva del mismo vehículo	Integridad de datos	Pruebas de concurrencia en entorno controlado
H3	El administrador accede a reportes mensuales en menos de 2 minutos	Tiempo de gestión	Medición durante pruebas con usuario real
H4	Los empleados operan el sistema con fluidez tras una sesión de capacitación de 30 minutos	Usabilidad	Puntaje SUS $\geq$ 70/100
H5	El sistema mantiene la integridad de los datos ante fallos inesperados	Confiabilidad	Pruebas de interrupción forzada con ROLLBACK

4.3 Supuestos de Diseño

Los supuestos son condiciones que se asumen como verdaderas para el funcionamiento del sistema. Si alguno no se cumple, representa un riesgo identificado:

Supuesto	Base	Riesgo si no se cumple
La empresa dispone de al menos un equipo con Windows instalado	Contexto habitual de empresas en SCZ	El sistema requiere adaptación para otro sistema operativo
Existe una conexión de red local entre los equipos de la empresa	Infraestructura básica de oficina	MySQL no podría compartirse entre equipos
Los empleados tienen conocimientos básicos de computación	Perfil laboral estándar	Se requiere capacitación adicional antes del uso
La empresa cuenta con al menos 3 vehículos en su flota	Tamaño mínimo viable para justificar el sistema	El sistema funcionaría pero su impacto sería mínimo
MySQL puede instalarse en el equipo servidor de la empresa	Software libre, sin costo de licencia	Restricciones de IT podrían impedirlo

5 OBJETIVOS

5.1 Objetivo General

Desarrollar un sistema de escritorio para la gestión integral de alquiler de vehículos en empresas del área urbana de Santa Cruz de la Sierra, que centralice el control de flota, clientes, reservas y pagos en una sola herramienta digital confiable, evaluando su viabilidad técnica y usabilidad mediante pruebas con usuarios reales.

5.2 Objetivos Específicos

- Identificar los procesos operativos críticos de empresas de alquiler vehicular en SCZ mediante encuestas y entrevistas para establecer los requisitos funcionales del sistema
- Diseñar el modelo de base de datos relacional con su Diagrama Entidad-Relación, normalizado a 3NF e implementado en sintaxis MySQL
- Construir los módulos de gestión de flota, clientes, reservas y pagos como núcleo funcional del sistema
- Incorporar mecanismos de seguridad basados en autenticación por roles, encriptación de contraseñas y registro de auditoría de acciones
- Generar reportes operativos y administrativos que permitan al administrador tomar decisiones basadas en datos reales de la flota e ingresos
- Validar la integridad de los datos del sistema mediante pruebas de concurrencia, interrupciones forzadas y verificación de restricciones en la base de datos

6 METODOLOGÍA

6.1 Tipo y Enfoque de Investigación

La investigación que sustenta el desarrollo de MOVISHARE tiene un enfoque **mixto**:

- **Cualitativo:** para entender cómo operan actualmente las empresas de alquiler vehicular en SCZ, qué problemas enfrentan en su día a día y qué esperan de una herramienta digital. Esto se recoge mediante entrevistas y observación directa.
- **Cuantitativo:** para medir el impacto real del sistema una vez implementado tiempos de operación, puntajes de usabilidad, métricas de rendimiento. El tipo de investigación

es tecnológica-proyectiva: no se limita a describir un problema sino que propone y construye una solución concreta para resolverlo.

6.2 Instrumento de Recolección

- **Entrevista semi-estructurada:**

Dirigida al propietario o administrador de la empresa de alquiler vehicular. Su objetivo es entender el flujo operativo actual — cómo registran reservas, cómo controlan su flota, qué problemas enfrentan — para traducir esa realidad en los requisitos funcionales del sistema. Los resultados de esta entrevista son la base del Product Backlog y la priorización de módulos.

6.3 Metodología de Desarrollo — Scrum

¿Por qué Scrum?

Scrum fue seleccionado como metodología de desarrollo por tres razones concretas aplicadas al contexto de MOVISHARE:

- **Primero**, el proyecto tiene módulos bien diferenciados flota, clientes, reservas, pagos, seguridad, reportes que se pueden construir y entregar de forma independiente en cada sprint, sin necesidad de terminar todo el sistema para tener algo funcional que mostrar.
- **Segundo**, durante el desarrollo es probable que el tutor o la empresa sugieran ajustes. Scrum permite incorporar esos cambios al inicio del siguiente sprint sin romper lo ya construido.

- **Tercero**, cada sprint genera evidencia documentada de avance — qué se planificó, qué se construyó, qué se entregó — lo que facilita la evaluación del tribunal académico.

6.4 Roles en el Proyecto

Rol	Responsabilidad	Quién
Product Owner	Define prioridades y valida entregables	Representante de la empresa / tutor
Scrum Master	Facilita el proceso y elimina bloqueos	Tutor académico
Equipo de desarrollo	Diseña, implementa y prueba	Desarrollador del proyecto

6.5 CONTEXTO LEGAL Y ECONÓMICO

6.5.1 Marco Legal del Alquiler Vehicular en Bolivia

Bolivia no cuenta con una ley específica que regule el alquiler de vehículos entre empresas y clientes particulares. Sin embargo, existen normas del ordenamiento jurídico general que enmarcan y validan esta actividad:

- **Código Civil Boliviano — Contrato de Arrendamiento (Art. 685 y 1297):**
El alquiler de un vehículo entre una empresa y un cliente es un contrato de arrendamiento privado, plenamente válido y reconocido por el Código Civil boliviano. No requiere autorización especial del Estado para celebrarse, siempre que ambas partes sean mayores de edad y el acuerdo sea voluntario.
- **Ley N° 3988 — Código de Tránsito:**
Distingue claramente entre vehículo particular (Art. 29) y vehículo de servicio público (Art. 30). Los vehículos de una empresa de alquiler operan como particulares

cedidos temporalmente a un cliente, no como transporte público, por lo que no requieren habilitación municipal como servicio de transporte.

• **RUAT** **Registro** **Único** **Automotor:**

Todo vehículo en Bolivia debe estar registrado en el RUAT, que certifica su propietario legal, su estado técnico y si tiene algún impedimento legal (embargo, litigio). MOVISHARE puede usar esta referencia para validar que los vehículos registrados en el sistema pertenecen efectivamente a la empresa.

- **Conclusión legal:** MOVISHARE opera en un marco legal favorable. El alquiler vehicular empresa-cliente es un contrato privado válido bajo el Código Civil boliviano, sin restricciones específicas que limiten su operación en el área urbana de Santa Cruz de la Sierra.

6.6 Mercado de Alquiler Vehicular en Santa Cruz de la Sierra

El mercado de alquiler vehicular en Santa Cruz de la Sierra presenta una brecha clara entre la oferta formal disponible y la demanda de usuarios que buscan opciones más accesibles y flexibles.

Precios del mercado actual :

Segmento	Precio estimado	Perfil del cliente
Agencias formales (turismo/empresas)	\$40 - \$130 / día	Turistas, ejecutivos, empresas
Alquiler informal entre conocidos	Variable, sin garantía	Residentes locales
MOVISHARE (objetivo)	Bs 150 - 400 / día	Residentes, trabajadores, estudiantes en SCZ

Características del mercado local

El mercado cruceño de alquiler vehicular se caracteriza por tres aspectos relevantes para MOVISHARE:

- **Alta informalidad:** una parte significativa de los alquileres en SCZ ocurre de forma informal, sin contrato, sin registro y sin garantías para ninguna de las partes. MOVISHARE digitaliza y formaliza ese proceso dentro de la empresa, generando contratos implícitos respaldados por registros en el sistema.
- **Ausencia de sistemas especializados:** la mayoría de las empresas pequeñas de alquiler en SCZ no utilizan software especializado. Gestionan su operación con cuadernos, hojas de cálculo o aplicaciones genéricas que no fueron diseñadas para este rubro. Esa brecha tecnológica es exactamente el espacio que MOVISHARE ocupa.
- **Demanda creciente:** el crecimiento urbano de Santa Cruz de la Sierra genera una demanda sostenida de movilidad flexible. Personas que no tienen vehículo propio, empresas que necesitan transporte temporal para empleados o eventos, y visitantes que requieren movilidad local representan un mercado activo y en expansión.

6.7 Sprints

Sprint	Objetivo	Entregable
Sprint 0	Levantamiento de requisitos, DER y arquitectura definida	Product Backlog + DER + Script SQL + documento de arquitectura
Sprint 1	Base de datos implementada en MySQL	Todas las tablas creadas con constraints e índices
Sprint 2	Módulo de gestión de flota	CRUD de vehículos + control de estado + mantenimiento
Sprint 3	Módulo de clientes	Registro, búsqueda e historial de clientes
Sprint 4	Módulo de reservas	Registro, devoluciones, vencidos y disponibilidad
Sprint 5	Módulo de pagos y tarifas	Cálculo automático + registro de pagos + pendientes
Sprint 6	Seguridad, reportes y auditoría	Login + roles + reportes + logs de auditoría
Sprint 7	Pruebas y evaluación final	Informe SUS + métricas de rendimiento + correcciones

6.8 ARQUITECTURA TÉCNICA

6.8.1 Modelo de Arquitectura

MOVISHARE sigue una arquitectura de **tres capas** (Three-Tier Architecture), estándar en sistemas de escritorio empresariales. Esta arquitectura separa el sistema en tres niveles independientes, cada uno con una responsabilidad clara:

6.8.2 ¿Por qué arquitectura de tres capas?

Ventaja	Explicación aplicada a MOVISHARE
Separación de responsabilidades	La interfaz no accede directamente a MySQL, siempre pasa por la lógica del controlador
Seguridad	Las validaciones y el control de roles ocurren en la capa lógica antes de llegar a la BD
Mantenibilidad	Si se cambia la interfaz gráfica, la lógica y la BD no se tocan
Facilidad de pruebas	Cada capa se puede probar de forma independiente

6.9 Stack Tecnológico

El stack tecnológico de MOVISHARE fue seleccionado priorizando herramientas de código abierto, compatibilidad con Windows y facilidad de mantenimiento a largo plazo. Cada componente cumple un rol específico dentro de la arquitectura de tres capas:

Capa	Tecnología	Versión	Función
Lenguaje	Python	3.10+	Lenguaje principal del sistema
Interfaz gráfica	CustomTkinter	5.2.2	Ventanas y formularios de escritorio con soporte oscuro/claro
Base de datos	MySQL	8.0+	Almacenamiento y gestión de datos
Conexión a BD	mysql-connector-python	9.7.0	Pool de conexiones entre Python y MySQL
Seguridad	bcrypt	4.1.3	Encriptación de contraseñas (NIST compliant)
Reportes / Dashboard	matplotlib	3.9.4	Gráficos embebidos en la interfaz (barras, pastel)
Variables de entorno	python-dotenv	1.2.2	Gestión segura de credenciales MySQL
Control de versiones	Git	—	Historial de cambios del código
Repositorio remoto	GitHub	—	Respaldo remoto del código fuente
IDE	VS Code o PyCharm	—	Entorno de desarrollo
Gestor de paquetes	pip	—	Instalación de librerías Python
Gestor de BD visual	MySQL Workbench	—	Administración visual de MySQL

Las credenciales de conexión a MySQL se gestionan mediante un archivo `.env` en la raíz del proyecto, excluido del repositorio por seguridad mediante `.gitignore`. Esto permite que cada miembro del equipo configure sus propias credenciales locales sin exponer datos sensibles en el código fuente.

6.10 Patrón de Diseño — MVC

MOVISHARE aplica el patrón Modelo-Vista-Controlador (MVC) dentro de su arquitectura de tres capas. Cada componente tiene una responsabilidad estricta y una restricción de diseño que garantiza la separación de intereses:

Componente	Responsabilidad	Restricción	Ubicación
Modelo	Consultas SQL parametrizadas, transacciones atómicas, retorno de datos puros	No importa tkinter ni customtkinter	Carpeta models/
Vista	Renderizado de widgets CustomTkinter, eventos de usuario, tablas scrollables	No contiene SQL	Carpeta views/
Controlador	Lógica de negocio, validaciones, encriptación bcrypt	Puente entre Modelo y Vista	Carpeta controllers/

El pool de conexiones MySQL se inicializa una sola vez al arrancar la aplicación en `main.py` mediante la clase `Database`, y se libera al cerrar la ventana. Esta clase actúa como pool centralizado con los métodos `execute_query` y `execute_many`, garantizando que toda operación con la base de datos pase por un único punto de acceso con manejo automático de `commit` y `rollback`. La configuración de conexión (host, usuario, contraseña, nombre de base de datos) se lee desde variables de entorno al momento de la inicialización.

6.11 Roles del Sistema y Control de Acceso

MOVISHARE define tres roles de usuario almacenados en una tabla catálogo independiente (rol), lo que permite agregar nuevos roles sin modificar la estructura de la base de datos:

Rol	Descripción	Acceso principal
ADMINISTRADOR	Acceso total al sistema	Flota, clientes, reservas, pagos, reportes y auditoría
OPERADOR	Gestión diaria de alquileres	Reservas, clientes y operación cotidiana
TÉCNICO	Mantenimiento de flota	Módulo de mantenimiento y gestión de vehículos

La autenticación se realiza mediante username y contraseña hasheada con bcrypt. Al iniciar sesión, el sistema verifica que el usuario exista, que su estado sea ACTIVO, y que la contraseña coincida con el hash almacenado. Cada inicio de sesión exitoso queda registrado automáticamente en la tabla de auditoría.

Función	Operador	Técnico	Administrador
Registrar / buscar cliente	✓	✗	✓
Registrar / ver vehículos	✓	✓	✓
Crear y gestionar reservas	✓	✗	✓
Registrar pagos	✓	✗	✓
Programar mantenimientos	✗	✓	✓
Ver reportes e ingresos (KPIs)	✗	✗	✓
Ver auditoría del sistema	✗	✗	✓
Crear cuentas de usuarios	✗	✗	✓

6.12 ANÁLISIS DE RIESGOS Y SEGURIDAD

Todo sistema de software que maneja información sensible de una empresa — datos de clientes, registros de pagos, historial de operaciones — está expuesto a riesgos. Identificarlos con anticipación y diseñar estrategias de mitigación concretas es tan importante como construir las funcionalidades del sistema.

En MOVISHARE los riesgos se clasifican en dos categorías: técnicos, relacionados con el comportamiento del software y la base de datos, y operativos, relacionados con el uso humano del sistema en el día a día de la empresa.

6.12.1 Clasificación de Riesgos

Cada riesgo se evalúa según probabilidad (Alta, Media, Baja) e impacto (Crítico, Moderado, Menor).

Probabilidad:

- Alta — ocurre frecuentemente en sistemas similares
- Media — puede ocurrir en condiciones específicas
- Baja — poco probable pero posible

Impacto:

- Crítico — detiene el sistema o corrompe datos
- Moderado — afecta la operación pero es recuperable
- Menor — incomodidad sin consecuencias graves

6.12.2 Riesgos Técnicos

R1 — Pérdida de conexión a MySQL durante una transacción

Dimensión	Valor
Probabilidad	Media
Impacto	Crítico
Escenario	El empleado registra una reserva y MySQL falla a mitad del proceso. La reserva queda insertada pero el vehículo no cambió de estado.
Mitigación	Pool de conexiones centralizado (Database.get_connection) con autocommit desactivado. Cada operación transaccional usa BEGIN/COMMIT/ROLLBACK manual. Si algo falla antes del commit, se ejecuta rollback automáticamente.

Código implementado en reserva_model.py:

```
conn = Database.get_connection()
cursor = conn.cursor()
try:
    cursor.execute(insert_reserva)
    cursor.execute(insert_reserva_automovil)
    cursor.execute(update_estado_vehiculo)
    conn.commit()
except Exception as e:
    conn.rollback()
    raise e
finally:
    cursor.close()
    conn.close()
```

R2 — Dos empleados reservan el mismo vehículo simultáneamente

Dimensión	Valor
Probabilidad	Media
Impacto	Crítico
Escenario	Dos empleados en equipos distintos ven el mismo vehículo disponible y crean una reserva al mismo tiempo.
Mitigación	El sistema valida el estado del vehículo al momento de crear la reserva. Al hacer UPDATE automovil SET estado='ALQUILADO', si el vehículo ya fue cambiado por otra transacción concurrente, el segundo intento encontrará un conflicto de estado. Adicionalmente, la lista de vehículos disponibles se filtra por estado='DISPONIBLE' al momento de seleccionar.

R3 — Pérdida o corrupción de la base de datos

Dimensión	Valor
Probabilidad	Baja
Impacto	Crítico
Escenario	Fallo del disco duro, corte de luz prolongado o eliminación accidental de la base de datos.
Mitigación	Se recomienda configurar un backup diario mediante script que exporte la BD con mysqldump. Los archivos se guardan localmente con fecha en el nombre.

R4 — Empleado accede a módulos restringidos

Dimensión	Valor
Probabilidad	Media
Impacto	Moderado
Escenario	Un operador o técnico intenta acceder a reportes o auditoría que no le corresponden.
Mitigación	El sistema almacena el rol del usuario autenticado (id_rol y nombre_rol) en el objeto de sesión. El menú lateral de la ventana principal puede filtrar las opciones visibles según el rol del usuario logueado.

6.12.3 Riesgos Operativos

R5 — Empleado registra fechas incorrectas en una reserva

Dimensión	Valor
Probabilidad	Alta
Impacto	Moderado
Escenario	El empleado ingresa una fecha de inicio pasada o una fecha final anterior a la de inicio.
Mitigación	El controlador valida las fechas antes de enviar a la BD: verifica formato YYYY-MM-DD, que fecha_final no sea anterior a fecha_inicio, y que la diferencia de días sea positiva. El precio se recalcula automáticamente al cambiar las fechas.

R6 — Precio total calculado incorrectamente

Dimensión	Valor
Probabilidad	Baja
Impacto	Moderado
Escenario	El cálculo de días o la obtención de tarifas falla por un error de formato o datos faltantes en las tablas catálogo.
Mitigación	El sistema calcula el precio usando la fórmula $(tarifa_diaria + seguro) \times días + costo_ámbito$ directamente desde las tablas CATEGORIA y AMBITO_USO. El controlador muestra el precio calculado antes de confirmar, permitiendo verificación visual del empleado. Si algún dato falta, retorna un mensaje de error descriptivo.

6.13 IMPACTO ESPERADO

6.13.1 Impacto Operativo

Este es el impacto más directo y verificable del proyecto. Compara la situación actual de una empresa de alquiler vehicular en SCZ que opera sin sistema, contra la situación con MOVISHARE implementado.

Situación actual sin MOVISHARE:

Una empresa que gestiona su operación manualmente enfrenta estos problemas cotidianos:

- Registros en cuadernos o Excel sin validación ni control

- No hay forma rápida de saber qué vehículos están disponibles en tiempo real
- Los errores de fechas o doble reserva se detectan cuando el cliente ya llegó
- No existe historial organizado de clientes ni de pagos
- El administrador no puede saber qué hizo cada empleado ni cuándo
- Los reportes se elaboran manualmente al final del mes

Situación con MOVISHARE

Proceso	Sin sistema	Con MOVISHARE
Verificar disponibilidad de vehículo	Revisar cuaderno o preguntar	Consulta en tiempo real, segundos
Registrar una reserva nueva	5-10 min manual con riesgo de error	2 min con validación automática
Calcular precio total del alquiler	Manual, propenso a error	Automático al seleccionar fechas
Saber qué vehículos están en mantenimiento	Depende de que alguien lo anote	Estado visible en el sistema
Generar reporte mensual	Horas de trabajo manual	Minutos con filtro por fecha
Detectar reservas vencidas	Revisión manual diaria	Visible en dashboard automáticamente
Rastrear qué empleado hizo qué	Imposible	Auditoría completa por usuario

6.13.2 Impacto Económico

Una gestión ineficiente genera pérdidas económicas concretas para la empresa. MOVISHARE las reduce de la siguiente forma:

Problema económico	Causa sin sistema	Solución con MOVISHARE
Vehículo alquilado dos veces	Sin control de disponibilidad en tiempo real	Estado sincronizado + SELECT FOR UPDATE
Cobro incorrecto al cliente	Cálculo manual de días y precio	Cálculo automático precio x días
Vehículo en mantenimiento alquilado	Empleado no sabía el estado	Estado visible y bloqueado en reservas
Pérdida de historial de pagos	Registros en papel sin respaldo	BD con backup diario automático
Tiempo de empleado en tareas administrativas	Reportes manuales, búsquedas en cuadernos	Automatizado, empleado enfocado en atención

6.13.3 Impacto en la Seguridad de la Información

Los datos que maneja una empresa de alquiler vehicular son sensibles: nombres, cédulas de identidad, teléfonos, direcciones y registros de pagos de sus clientes. Sin un sistema especializado esa información está expuesta a pérdida, acceso no autorizado o manipulación.

Con MOVISHARE:

- Las contraseñas se almacenan con bcrypt, no en texto plano
- Solo usuarios autenticados acceden al sistema
- Cada acción queda registrada en AUDITORIA con usuario, fecha y hora
- Las eliminaciones críticas requieren confirmación explícita
- Los backups diarios garantizan recuperación ante pérdida de datos

6.13.4 Impacto Académico

Desde la perspectiva de la ingeniería de sistemas, MOVISHARE demuestra la aplicación integrada de los conocimientos adquiridos durante la carrera:

Conocimiento	Aplicación en MOVISHARE
Bases de datos relacionales	9 tablas normalizadas a 3NF en MySQL
Modelado de datos	DER completo con entidades, relaciones y cardinalidades
Programación orientada a objetos	Arquitectura MVC en Python con patrón Singleton
Seguridad informática	bcrypt, control de sesión, auditoría, manejo de excepciones
Metodología de desarrollo	Scrum con 8 sprints y entregables verificables

7 CONCLUSIONES

a. Conclusiones Generales

MOVISHARE surge como respuesta a una necesidad real y concreta: empresas de alquiler vehicular en el área urbana de Santa Cruz de la Sierra que gestionan su operación de forma manual, con los errores, pérdidas y limitaciones que eso implica. El proyecto demuestra que una solución tecnológica bien diseñada, adaptada al contexto local y construida con rigor técnico, puede transformar significativamente la eficiencia operativa de ese tipo de empresa.

A lo largo del desarrollo del documento se estableció un hilo conductor claro entre el problema identificado, las hipótesis planteadas, los objetivos definidos y las decisiones técnicas tomadas. Cada módulo del sistema, cada tabla de la base de datos y cada mecanismo de seguridad responde directamente a una necesidad levantada en el análisis de requisitos.

b. Limitaciones del Proyecto

Limitación	Explicación
Alcance local	El sistema está diseñado para una sola empresa con servidor local, no es multisucursal ni web
Sin pago en línea	El módulo de pagos registra transacciones pero no procesa pagos digitales en tiempo real
Muestra de prueba reducida	Las pruebas SUS se realizan con 5-10 usuarios beta, lo que limita la generalización estadística
Dependencia de MySQL local	Requiere que MySQL esté instalado y corriendo en el equipo o red local de la empresa
Sin app móvil	Es exclusivamente de escritorio, los clientes no pueden hacer reservas desde su celular

Estas limitaciones no invalidan el proyecto sino que definen claramente su alcance y abren líneas de trabajo futuro.

c. Trabajo Futuro

A partir de MOVISHARE se identifican las siguientes extensiones naturales del sistema:

- Versión web accesible desde cualquier dispositivo y navegador
- App móvil para que los clientes consulten disponibilidad y reserven desde su celular
- Integración con pasarelas de pago locales bolivianas
- Módulo de seguimiento GPS de la flota en tiempo real
- Adaptación multisucursal para empresas con más de una sede
- Notificaciones automáticas por WhatsApp o SMS a clientes sobre su reserva

8 RECOMENDACIONES

a) Para el Desarrollo del Sistema

- **Empezar por la base de datos:**

Antes de escribir una sola línea de interfaz gráfica, la base de datos debe estar completamente implementada, probada y con datos de prueba cargados. Todo el sistema depende de ella y cualquier cambio tardío en el modelo de datos genera trabajo extra en todas las capas.

- **Respetar la arquitectura MVC desde el inicio:**

Es tentador mezclar lógica de negocio dentro de los archivos de la vista para avanzar más rápido. Ese atajo genera deuda técnica que después es difícil de corregir. Mantener la separación estricta entre vista, controlador y modelo desde el sprint 1 ahorra tiempo a largo plazo.

- **Probar cada módulo antes de pasar al siguiente:**

Scrum define una Definición de Terminado (DoD) por razón. Un módulo que no está completamente probado y aprobado no debe considerarse cerrado aunque visualmente funcione. Los errores no detectados en sprint 2 se convierten en problemas complejos en sprint 5.

- **Hacer commits frecuentes en Git:**

Cada avance significativo — una función nueva, una tabla creada, un bug corregido — debe quedar registrado en Git con un mensaje descriptivo. Esto protege el trabajo ante pérdidas accidentales y permite volver a versiones anteriores si algo sale mal.

b) Para la Empresa que Adopte el Sistema

- **Capacitar al personal antes del lanzamiento:**

El sistema más bien diseñado fracasa si los usuarios no saben usarlo. Se recomienda una sesión de capacitación de al menos 30 minutos con cada empleado antes de la puesta en producción, cubriendo los flujos más frecuentes: registrar reserva, registrar devolución y cobrar pago.

- **Definir un responsable del sistema:**

Debe existir una persona designada — preferentemente el administrador — que sea el punto de contacto para dudas, errores y actualizaciones. Sin un responsable claro, los problemas tienden a ignorarse hasta que se vuelven críticos.

- **No saltarse el backup diario:**

El backup automático es la última línea de defensa ante pérdida de datos. Se recomienda verificar periódicamente que los archivos de backup se estén generando correctamente y almacenar una copia adicional en un dispositivo externo o servicio en la nube.

c) **Para Futuras Versiones del Sistema**

- **Migrar a arquitectura web en la siguiente versión:**

El sistema de escritorio es adecuado para el alcance actual, pero una versión web permitiría acceso desde cualquier dispositivo, facilitaría las actualizaciones y eliminaría la dependencia de instalación local. Flask o FastAPI con React serían opciones naturales dado el stack Python actual.

- **Implementar un módulo de estadísticas:**

Los datos que MOVISHARE acumula durante su operación — vehículos más alquilados, períodos de mayor demanda, clientes más frecuentes — son valiosos para

la toma de decisiones. Un módulo de estadísticas visual convertiría esos datos en información accionable para el administrador.

- **Considerar integración con RUAT:**

Si en el futuro existe una API pública del RUAT boliviano, integrarla al sistema permitiría verificar automáticamente la propiedad y el estado legal de los vehículos registrados, fortaleciendo la confiabilidad del inventario de flota.

9 BIBLIOGRAFÍA

a) Libros y Textos Académicos

- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387.
- Date, C. J. (2004). *An Introduction to Database Systems* (8va ed.). Pearson Addison Wesley.
- Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería del Software: Un Enfoque Práctico* (8va ed.). McGraw-Hill.
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7ma ed.). McGraw-Hill.
- Sommerville, I. (2016). *Ingeniería de Software* (10ma ed.). Pearson.
- Laudon, K. C., & Laudon, J. P. (2020). *Sistemas de Información Gerencial* (16va ed.). Pearson.
- NIST (2018). *Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1*. National Institute of Standards and Technology.

b) Documentación Técnica Oficial

- MySQL (2024). *MySQL 8.0 Reference Manual*. Oracle Corporation.
- Python Software Foundation (2024). *Python 3.11 Documentation*.
- CustomTkinter (2024). *CustomTkinter Documentation*.
- mysql-connector-python (2024). *MySQL Connector/Python Developer Guide*.
- bcrypt (2024). *bcrypt — Password Hashing Library*.
- Git (2024). *Git Documentation*.
- GitHub (2024). *GitHub Documentation*.

c) Marco Legal

- Estado Plurinacional de Bolivia (2010). *Código Civil Boliviano — Artículos 685 y 1297: Contrato de Arrendamiento*. Gaceta Oficial de Bolivia.
- Estado Plurinacional de Bolivia (2011). *Ley N° 3988 — Código de Tránsito*. Gaceta Oficial de Bolivia.
- Registro Único Automotor — RUAT Bolivia (2024). *Consulta de vehículos registrados*.

10 ANEXO

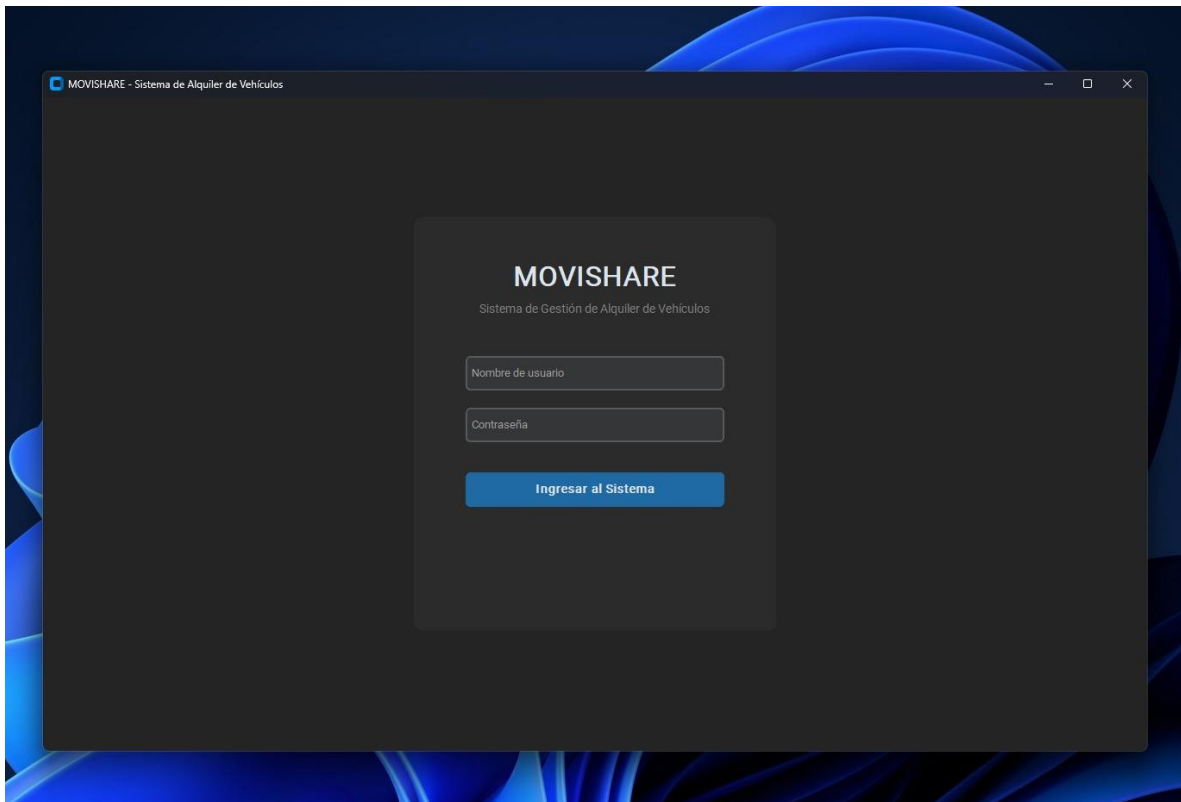


Figura 1. *Panel de inicio de usuario*

MOVISHARE

Usuario: admin

Rol: Administrador

Gestión de Flota

Gestión de Clientes

Reservas y Pagos

Reportes y Auditoría

Cerrar Sesión

Registro de Cliente

C.I.:

Nombre Completo:

Teléfono:

Dirección:

Email:

Datos de Licencia de Conducir

Nro. Licencia:

Categoría:

Fecha Vencimiento: YYYY-MM-DD

Nuevo

Guardar

Eliminar

Listado de Clientes

ID	C.I.	Nombre	Teléfono	Email	Licencia
7	5678901	Diana Torres Paredes	77756789	diana.torres@email.com	LIC-2024-078
6	3456789	Luis Fernández Castro	77734567	luis.fernandez@email.com	LIC-2022-1
5	7890123	Ana Martínez Ruiz	77778901	ana.martinez@email.com	LIC-2023-04
2	1234567	Juan Pérez García	77712345	juan.perez@email.com	LIC-2024-001

Figura 2. Modulo de registro

MOVISHARE

Usuario: admin

Rol: Administrador

Gestión de Flota

Gestión de Clientes

Reservas y Pagos

Reportes y Auditoría

Cerrar Sesión

Reservas de Alquiler

Control de Pagos

Contrato de Alquiler

Cliente:

5 - Ana Martínez Ruiz

Usuario:

Cargando...

Ámbito:

5 - EVENTO CORPORATIVO

Fecha Inicio:

YYYY-MM-DD

Fecha Final:

YYYY-MM-DD

Precio Total (Bs):

Calcule o ingrese

Litros Gas. Salida:

Km Salida:

Litros Gas. Retorno:

Km Retorno:

Estado:

PENDIENTE

Vehículo:

1 - ABC1234

Nuevo

Calcular

Guardar

Eliminar

Listado de Reservas

Nro	Cliente	Usuario	Inicio	Final	Precio	Estado
5	Juan Pérez García	admin	2025-03-08 00:00:00	2025-03-10 00:00:00	540.00	
4	Diana Torres Paredes	mgonzalez	2025-02-10 00:00:00	2025-02-15 00:00:00	950.00	
3	Luis Fernández Castro	admin	2025-07-01 00:00:00	2025-07-03 00:00:00	150.00	
2	Ana Martínez Ruiz	mgonzalez	2025-06-20 00:00:00	2025-06-25 00:00:00	810.00	
1	Juan Pérez García	admin	2025-01-15 00:00:00	2025-01-18 00:00:00	430.00	

Figura 3. Modulo de Contrato

46

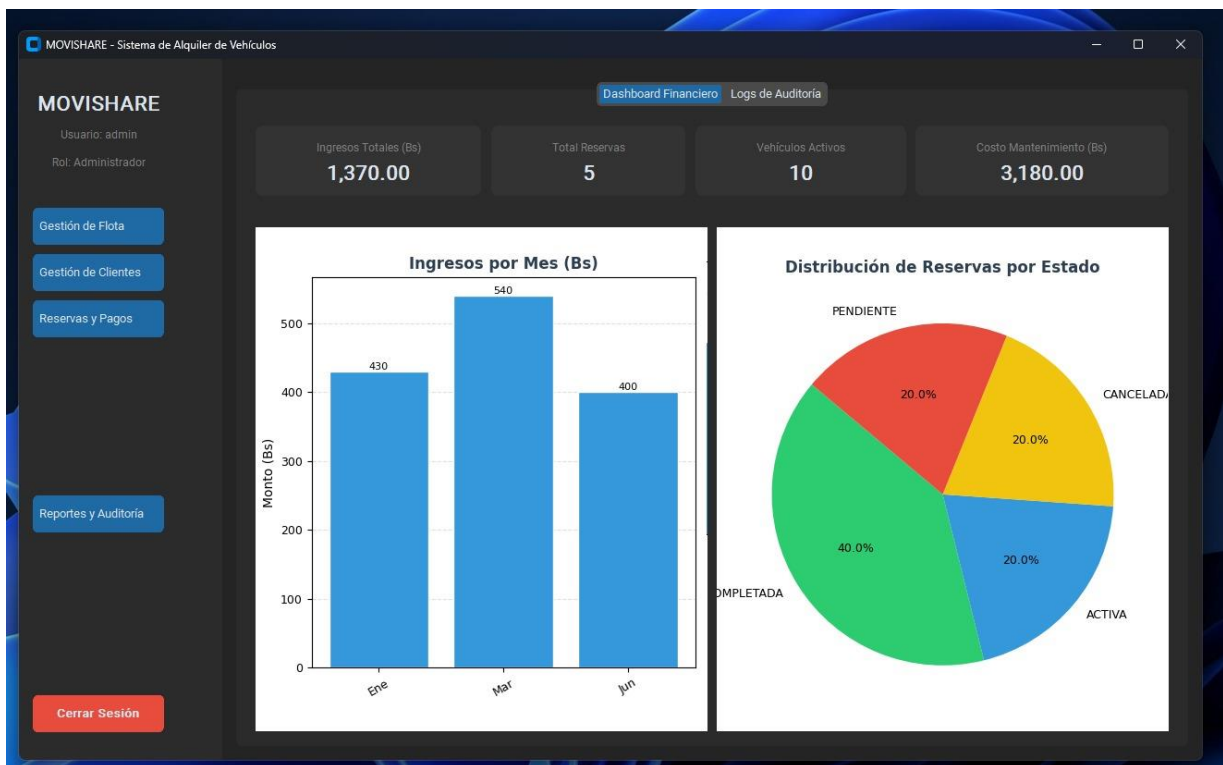


Figura 4. *Modulo Dashboard Financiero*

MOVISHARE - Sistema de Alquiler de Vehículos

Gestión de Vehículos | Control de Taller

Usuario: admin
Rol: Administrador

Datos del Vehículo

Placa:

Marca:

Modelo:

Año:

Color:

Kilometraje:

Estado:

Categoría:

Listado de Vehículos

ID	Placa	Marca	Modelo	Año	Estado	Categoría
10	YZA7890	Lincoln	Navigator	2022	DISPONIBLE	LIMOUSINE
9	VWX3456	Porsche	911	2023	ALQUILADO	DEPORTIVO
8	STU9012	Hyundai	H1	2020	DISPONIBLE	MINIVAN / VAN
7	PQR5678	Ford	Ranger	2021	MANTENIMIENTO	CAMIONETA C
6	MNO1234	Mercedes	GLE	2023	DISPONIBLE	SUV PREMIUM
5	JKL7890	Toyota	RAV4	2022	DISPONIBLE	Premium / SUV
4	GHI3456	BMW	Serie 3	2023	DISPONIBLE	Estándar (Sedan
3	DEF9012	Ford	Focus	2021	DISPONIBLE	Económico (Hatch
2	XYZ25678	Honda	CR-V	2022	DISPONIBLE	Premium / SUV
1	ABC1234	Toyota	Corolla	2023	DISPONIBLE	Económico (Hatch

Menú Lateral:

- Gestión de Flota
- Gestión de Clientes
- Reservas y Pagos
- Reportes y Auditoría
- Cerrar Sesión

Figura 5. *Modulo Datos del Vehículo*

Santa Cruz, 8 de junio de 2026

Señor
Lic. Edwin Vargas Yapura
Coordinador de Investigación de la FICCT
Presente.

REF. REVISIÓN DE 2DO-AVANCE DE PROYECTO DE INVESTIGACIÓN PARA LA FERIA FACULTATIVA.

De mi mayor consideración,

El Proyecto de Investigación Académica Formativa:

MATERIA: INF112-14

DOCENTE: Ing. Juan Carlos Peinado Pereira

TEMA: Plataforma de alquiler de vehículos entre particulares

ÁREA / CATEGORÍA: Desarrollo de software

INTEGRANTES:

Sara Raquel Sahonero Salas	224029096	Ing. Informática
Jarith Padilla Flores	224028537	Ing. Informática
Neil Serrano Vargas	220153892	Ing. Informatica
Diego Orellana Gutiérrez	224028464	Ing. Informatica
David Leonel Vincenty Mamani	224029584	Ing. Informatica

El documento de 2do-Avance (Título, Contexto, Planteamiento del Problema y Objetivos, Marco Teórico, Bibliografía), fue revisado en detalle, para participar en la Feria Facultativa, FICCT 1-2026, en representación de la Asignatura a mi cargo.

Sin otro particular, le reitero mis cordiales saludos.
Atentamente,



Ing. Juan Carlos Ruiz Garnica
Docente Tutor

Carta del docente (tutor)